Module 3 – Express.js (Detailed Notes)

1. Introduction to Express.js

Express.js is a minimal and flexible web application framework for Node.js. It provides a robust set of features for building web servers and APIs.

Key Features:

- Simplifies server-side development
- Handles HTTP requests and responses
- Supports middleware for modular functionality
- Ideal for building RESTful APIs

---

2. Setting Up Express

To use Express, it must first be installed and initialized in a Node.js project.

Installation:

```
npm install express
```

Basic Setup:

```
const express = require("express");
const app = express();

// Start server
app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Explanation:

- `express()` creates an application instance.
- `app.listen()` starts the server on a specified port.

---

3. Routing

Routing refers to defining how the server responds to client requests for specific endpoints.

Basic Route Syntax:

```
app.METHOD(PATH, HANDLER);
```

Common HTTP Methods:

GET (Retrieve Data)

```
app.get("/users", (req, res) => {
  res.send("Get all users");
});
```

POST (Create Data)

```
app.post("/users", (req, res) => {
  res.send("Create a new user");
});
```

PUT (Update Data)

```
app.put("/users/:id", (req, res) => {
  res.send("Update user");
});
```

DELETE (Remove Data)

```
app.delete("/users/:id", (req, res) => {
  res.send("Delete user");
});
```

Route Parameters:

- Used to capture dynamic values from the URL.

```
app.get("/users/:id", (req, res) => {
  console.log(req.params.id);
});
```

---

4. Middleware

Middleware functions are functions that execute during the request-response cycle before the final route handler.

Purpose of Middleware:

- Modify request (`req`) and response (`res`) objects
- Execute logic such as logging, authentication, validation
- Control flow using `next()`

Basic Middleware Example:

```
app.use((req, res, next) => {
  console.log("Request received");
  next();
});
```

Types of Middleware:

- Application-level middleware
- Router-level middleware
- Built-in middleware (e.g., `express.json()`)
- Third-party middleware (e.g., logging libraries)

Using Built-in Middleware:

```
app.use(express.json());
```

---

5. Authentication with JWT (JSON Web Tokens)

JWT is used to securely transmit information between client and server.

a. Generating a Token

```
const jwt = require("jsonwebtoken");

const token = jwt.sign(
  { userId: 1 },
  "secretKey",
  { expiresIn: "1h" }
);
```

b. Verifying a Token

```
jwt.verify(token, "secretKey", (err, decoded) => {
  if (err) {
    console.log("Invalid token");
  } else {
    console.log(decoded);
  }
});
```

c. Protecting Routes (Middleware)

```
const authMiddleware = (req, res, next) => {
  const token = req.headers.authorization;

  if (!token) {
    return res.status(401).send("Access denied");
  }

  try {
    const verified = jwt.verify(token, "secretKey");
    req.user = verified;
    next();
  } catch (err) {
    res.status(400).send("Invalid token");
  }
};
```

Usage:

```
app.get("/protected", authMiddleware, (req, res) => {
  res.send("Protected route");
});
```

---

6. Error Handling

Error handling ensures that the application responds properly when something goes wrong.

a. Centralized Error Middleware

- Defined at the end of all routes
- Uses four parameters: `(err, req, res, next)`

```
app.use((err, req, res, next) => {
  console.error(err.message);
  res.status(500).send("Internal Server Error");
});
```

b. Passing Errors

```
app.get("/", (req, res, next) => {
  const error = new Error("Something went wrong");
  next(error);
});
```

Benefits:

- Keeps code clean and organized
- Avoids repeating error-handling logic
- Improves debugging and maintainability

---

7. API Design Best Practices

Designing clean and maintainable APIs is essential for scalability.

a. Use RESTful Principles

- Use proper HTTP methods:
  - GET → retrieve data
  - POST → create data
  - PUT/PATCH → update data
  - DELETE → remove data

b. Consistent Naming

- Use clear and meaningful endpoint names
- Example:
  - `/users`
  - `/users/:id`

c. Use Proper Status Codes

- 200 → Success
- 201 → Created
- 400 → Bad Request
- 401 → Unauthorized
- 404 → Not Found
- 500 → Server Error

d. Input Validation

- Validate incoming data to prevent errors and security issues
- Use libraries like Joi or express-validator

e. Structure Your Project

- Separate routes, controllers, and middleware
- Keep code modular and organized

---

8. Key Concepts to Remember

- Express.js simplifies the creation of web servers and APIs.
- Routing defines how the server responds to different HTTP requests.
- Middleware allows reusable logic during request processing.
- JWT is used for secure authentication and authorization.
- Centralized error handling improves application stability.
- Following API design best practices ensures scalability and maintainability.

---

9. Summary

Express.js is a powerful backend framework that streamlines the development of server-side applications. With features like routing, middleware, authentication, and structured error handling, it provides everything needed to build secure and scalable APIs. Mastering Express is essential for developing robust backend systems in the MERN stack.